

Lab 1b: Building Interactive Components with GitHub Copilot (Bonus Lab - Optional)

Objective

Learn how to use GitHub Copilot to build interactive components from scratch. You'll create a complete application with styling and functionality, experiencing how Copilot can assist across multiple technologies in a single project.

Note: This is an optional bonus lab that builds upon the concepts from Lab 1. Complete this if you have extra time or want additional practice with Copilot across different file types.

What You Will Learn in This Lab

- Using Copilot to generate structure and markup in your preferred technology
- Getting styling suggestions for responsive design
- Creating interactive functionality with event handling
- Building a complete modal/dialog component with open/close functionality
- Understanding how Copilot handles multi-technology projects
- Best practices for structuring component code with AI assistance

Technology Options: This lab can be completed using:

- **Web Technologies:** HTML, CSS, JavaScript (examples shown)
- **React:** JSX components with CSS/styled-components
- **Vue:** Vue components with scoped CSS
- **Angular:** Angular components with TypeScript
- **Desktop:** WPF (C#), JavaFX (Java), or similar
- **Mobile:** React Native, Flutter, or native frameworks

Examples use web technologies, but adapt to your preferred stack!

Steps

Step 1: Project Setup

1. Create a new folder called `copilot-components-lab`
2. Set up files according to your technology choice: **Web Technologies:**
 - `index.html`, `styles.css`, `script.js`

React:

- `Modal.jsx`, `Modal.css` (or styled-components)

Vue:

- `Modal.vue` (single file component)

Angular:

- `modal.component.ts`, `modal.component.html`, `modal.component.css`

Other frameworks: Create component files as appropriate for your stack

Step 2: Generate Structure

1. Open your main markup/component file and start with this comment:

```
html

... <!-- Modern page with modal dialog component -->
... <!-- Include: header, main content area, modal overlay, and proper semantic structure -->
```

(Adapt comment syntax to your technology) 2. Type the opening structure for your technology and let Copilot suggest the rest 3. Add this comment and let Copilot generate the layout structure:

```
html

... <!-- Page layout: header with title, main content with button to open modal, footer -->
```

4. Add this comment for the modal:

```
html

... <!-- Modal component: overlay background, modal content box, close button, title, and body text -->
```

Step 3: Styling with Copilot

1. Open your styling file and add this comment:

```
css

... /* Modern, clean styling for modal component */
... /* Include: responsive design, smooth transitions, and accessibility focus states */
```

(Use appropriate comment syntax for your technology) 2. Start typing CSS reset or base styles and let Copilot suggest:

CSS

```
* {  
  margin: 0;  
  padding: 0;
```

3. Add styling for the modal with this comment:

CSS

```
/* Modal overlay - full screen, semi-transparent background */  
.modal-overlay {
```

4. Continue with modal content styling:

CSS

```
/* Modal content box - centered, white background, rounded corners, shadow */  
.modal-content {
```

Step 4: Interactive Functionality

1. Open your logic/script file and add this comment:

javascript

```
// Modal component functionality  
// Features: open modal, close modal, close on overlay click, close on escape key
```

(Adapt to your language syntax) 2. Start with element selection appropriate to your technology:

javascript

```
// Get modal elements  
const modal = document.querySelector(
```

3. Add event listeners with Copilot's help:

javascript

```
// Event listener to open modal when button is clicked  
openButton.addEventListener(
```

4. Create the close functionality:

javascript

```
// Function to close modal - remove active class and hide modal  
...function closeModal() {
```

Step 5: Enhanced Features

1. Add this comment in your logic file:

javascript

```
// Close modal when clicking outside the modal content
```

2. Let Copilot suggest the event delegation code

3. Add keyboard accessibility:

javascript

```
// Close modal when escape key is pressed  
...document.addEventListener('keydown', function(event) {
```

Step 6: Form Integration

1. Add this comment in your markup:

html

```
<!-- Contact form inside modal: name, email, message fields with submit button -->
```

2. In your logic file, add form handling:

javascript

```
// Handle form submission - prevent default, collect data, show success message  
...const form = document.querySelector(
```

Step 7: Animation and Polish

1. Add this comment in your styling:

css

```
/* Smooth animations for modal open/close */  
.../* Fade in overlay, slide in modal content */
```

2. Add transitions and animations with Copilot's help:

```
CSS  
...@keyframes modalFadeIn {
```

Step 8: Testing and Debugging

1. Run your application using appropriate method for your technology:

- **Web:** Open `index.html` in browser
- **React:** `npm start`
- **Vue:** `npm run serve`
- **Angular:** `ng serve`

2. Test all functionality:

- Click to open modal
- Close with X button
- Close by clicking overlay
- Close with Escape key
- Test form submission

3. Check responsive behavior by resizing the window

4. Use browser developer tools or appropriate debugging tools

Step 9: Accessibility Improvements

1. Add this comment in your logic file:

```
javascript  
  
// Accessibility: trap focus within modal when open, restore focus when closed
```

2. Let Copilot help you implement focus management

3. Add ARIA attributes with this comment in markup:

```
html
```

<!-- Add proper ARIA labels and roles for screen readers -->

Step 10: Code Review and Optimization

1. Review all the generated code
2. Look for any redundant styling or logic
3. Add this comment and see if Copilot can help optimize:

```
javascript
```

```
... // Refactor repetitive code into reusable functions
```

Key Takeaways to Discuss

- Copilot understands context across different file types in a project
- Semantic comments help generate better structure
- Styling comments describing visual intent lead to more appropriate suggestions
- Logic comments focusing on user interactions generate better event handling code
- Always test AI-generated code in your target environment
- Accessibility considerations can be enhanced with specific prompts

Challenge Extension

Try adding these features with Copilot's help:

- Multiple modal types (success, warning, error)
- Animation options (slide, bounce, fade)
- Touch interactions for mobile
- Theme switching (light/dark mode)

Next Steps

Experiment with different comment styles and see how they affect Copilot's suggestions. Try being more specific vs. more general in your descriptions.